

CS598: Cloud Computing Capstone

Project Task 1: Batch Processing Big Data in AWS

By: Chris Johnson

*University of Illinois at Urbana-Champaign
June 2020*

Task 1- Project Vision:

*Use Batch Processing in AWS to Efficiently Query a
Massive Airline Data Set (20+ GB)*

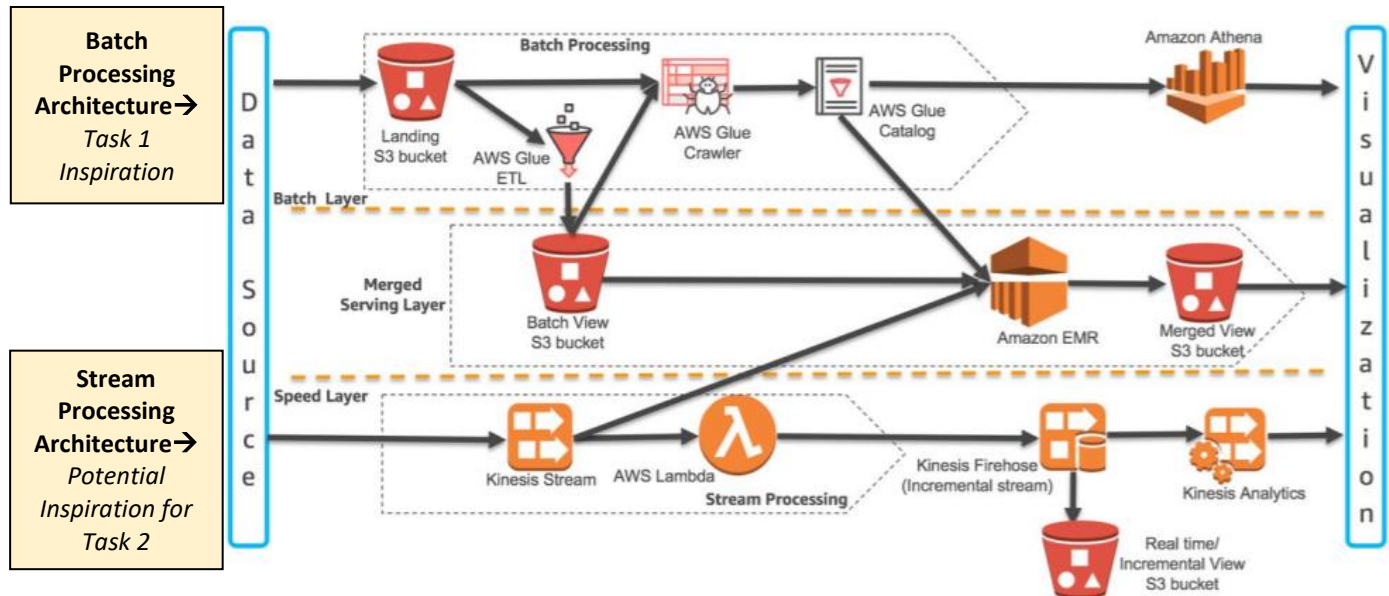
<i>Table of Contents.....</i>	<i>1</i>
<i>Task 1 Overview & System Design</i>	<i>2</i>
<i>Group 1 Questions</i>	<i>3</i>
<i>Group 2 Questions</i>	<i>4</i>
<i>Group 3 Questions</i>	<i>5</i>
<i>Appendix.....</i>	<i>7</i>

Task 1: Overview & System Design

Key Highlighted Areas: System Architecture & Diagram (Yellow), Three Major Setup & Integration Steps (Green), and Two System-Level Design Optimizations (Turquoise)

System Architecture & Diagram

My architecture choice was influenced by lessons learned in CS498 Cloud Computing Applications as well as AWS's 2018 white paper 'Lambda Architecture for Batch & Stream Processing' which concluded: "This suggested architecture on the AWS platform includes Amazon Athena for the batch layer."



Key Step #1: Move Raw Data into AWS S3

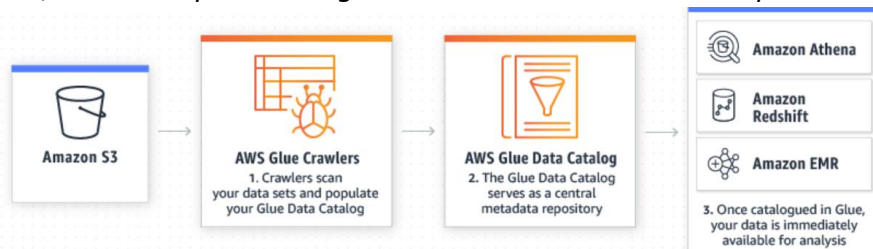
Move data from the public AWS 'EBS Snapshot' into an 'AWS S3' storage bucket.

Initially the data was hosted in an EBS snapshot, but I needed to move it to S3. First, I found the public EBS Snapshot, made a copy of my own, then created an EBS volume of the snapshot. Next, I launched Linux EC2 instance in the same AZ as the EBS volume and attached the EBS volume to this instance. Then, I transferred the files via:

- **SSH:** To SSH into the instance, I used Putty. To do so, I created a ppk key in PuttyGen.
- **Block Store:** I used command 'lsblk' to find the instance block stores, created a directory called 'data' on the instance via Putty/Ubuntu, then mounted the block store to the data directory so it was usable in Linux.
- **Transfer Files:** Next, I used WinSCP to copy the 'aviation' files from the transportation data to my local drive. I created a bucket in S3, then transferred only the 'airline_ontime' files to S3.

Key Step #2: Begin the ETL Process

Once data is stored in S3, use Hadoop-based engine 'AWS Glue' to start the ETL process.



Architecture: What is AWS Glue? AWS Glue, based on Hadoop & Spark, is a managed Extract, Transform, Load service. The AWS Glue Data Catalog lets you store and share metadata in a similar manner to Apache Hive.

Why is it a System-Level Optimization? AWS Glue is 'serverless', which means AWS handles the infrastructure portion- this gives me more time to focus on data analytics. It also can be more cost effective, as I basically only pay for what I

use when I use it. The crawler I created in AWS Glue went through the .csv files stored in S3, autogenerated the initial data schema, and associated the metadata with a new table in a new database.

Key Step #3: Explore & Clean Data, Then Perform Queries

Use Glue along with Apache Hadoop-based SQL query engine 'AWS Athena'.

Architecture: What is AWS Athena? AWS Athena uses Apache Hive DDL to define tables, is 'serverless', and uses AWS's version of Presto, an open-source distributed SQL query engine. In CS498 Cloud Computing Applications, Professor Farivar called Presto 'Interactive Hive on Steroids'.

Why is it a System-Level Optimization? Athena's familiar SQL syntax allows a quicker ramp up period and less trial-and-error. It is also 'serverless' like AWS Glue which eases administration and is cost-effective (pay per use).

Data Cleaning: I took an inventory of data fields, explored the data, and removed null records. There were 21 years of data on 116+ million flights, with 75 fields (17 complete years, 4 partial years).

- 1993 is missing Sept., 1994 is missing March, 2002 is missing May-Dec, 2008 is missing Nov-Dec.
- For size reasons, I only kept the columns needed for queries: *Year, Month, DayofMonth, DayOfWeek, FlightDate, UniqueCarrier, Origin, Dest, CRSDepTime, DepTime, DepDelayMinutes, ArrTime, ArrDelayMinutes, Cancelled*. Note took a different approach for Group 3 Q2, discussed in that section.
- For Arrival & Departures, there were multiple performance metrics and some had negative values. As a passenger would typically just care if departure/arrival was *on-time* (a value of 0) or *not* (value above 0). DepDelay & ArrDelay had negative values, so instead I used DepDelayMinutes & ArrDelayMinutes.

Data Input / Output Processing Format: org.apache.hadoop.hive.ql.io.parquet.MapredParquet

Serializer/ Deserializer Notes: I was having problems with the default 'LazySimpleSerDe', as some columns were being split in two. I switched to 'OpenCSV SerDe', then made custom selections to ensure the columns were being split correctly. For this dataset, I changed all the fields to 'string' types so that it would work with the Open CSV SerDe. The downstream impact is I need to specify the type in SQL queries using numeric values.

Task 1: Group 1 Questions

Group 1: Do My Results Make Sense & Are They Useful? Yes, the results make sense based on my experience with air travel, as I fly 50,000 to 100,000 miles per year for work. They are also in-line with the answers given by the professor.

- 1.1: Chicago (ORD), Atlanta (ATL), Dallas Fort-Worth (DFW), and Los Angeles (LAX) are by far the most popular airports as they are hubs. This gives users a good sense of where they will be connecting.
- 1.2: The average on-time arrival performance for the top 10 airlines was between 0 and 10 minutes, which is in-line with my expectations as well as the given answers. Note- I used DepDelayMinutes instead of DepDelay, because I did not want to include negative values in my calculation of on-time performance (i.e. early = on-time).

Group 1: Approach & Algorithms For both Group 1 questions, I used SQL queries on the cleaned data saved in S3. Given the schema, I had to use 'CAST' to convert strings to numeric values where appropriate on both. For Question 1.1, I had to use a 'JOIN' function so my counter function included flights both to & from the airports.

Question 1.1 Results: Rank the top 10 most popular airports by numbers of flights to/from the airport.

	origin ▼	counter ▼
1	ORD	12449353
2	ATL	11540422
3	DFW	10799303
4	LAX	7723595
5	PHX	6585534
6	DEN	6273787
7	DTW	5636622
8	IAH	5480734
9	MSP	5199213
10	SFO	5171033

Question 1.2 Results: Rank the top 10 airlines by on-time arrival performance.

	uniquecarrier ▼	AverageDelayMin ▼
1	HA	3.9723442136498517
2	AQ	5.044499594078162
3	PS	5.672566797335321
4	ML (1)	8.703598142334235
5	WN	9.128027655975494
6	F9	9.933341238438784
7	PA (1)	10.344808759846858
8	US	10.53142356392608
9	NW	10.584071568049064
10	OO	10.660752365340201

Task 1: Group 2 Questions

Group 2: Do My Results Make Sense & Are They Useful? Yes, as they allow users to choose the best performing airlines for several popular airports in the US. They also show average delays for several key source-destination pairs. The results are in-line with the answers given by the professor as well (i.e. within 10% target). Notes:

- 2.1: There was a tight band of departure performance across the stated airports (i.e. 0 to 12 minutes). I used DepDelayMinutes instead of DepDelay, because I did not want to negative values to impact my calculations.
- 2.3 & 2.4: The average delay performance for the top 10 carriers at the given source-destination pairs was between 5 and 19 minutes. The mean arrival delay for all the source-destination pairs was between 6 and 16 minutes. I used ArrDelayMinutes instead of ArrDelay, because I did not want negative values to impact results.

Group 2: Approach & Algorithms For all Group 2 questions, I used SQL queries on the cleaned data saved in S3. Given the schema, I had to use 'CAST' to convert strings to numeric values where appropriate on both. For Questions 2.3 and 2.4, I performed several queries on the smaller dataset to obtain the results for the 6 different source-destination pairs.

Question 2.1 Results: For each airport X, rank the top-10 carriers in decreasing order of on-time departure performance from X.

CMI

uniquecarrier	AverageDepDelayMin
1 US	2.8827454718779792
2 TW	4.158153846153846
3 PI	4.522930315664086
4 OH	5.364254792826221
5 DH	9.649402390438247
6 EV	9.692660550458715
7 MQ	11.754489920586439

BWI

uniquecarrier	AverageDepDelayMin
1 F9	4.916083916083916
2 PA (1)	5.942857142857143
3 CO	7.1413334153013865
4 AA	7.657054909239057
5 YV	7.675990675990676
6 NW	8.30940419738016
7 US	8.514172363028138
8 DL	8.81506807645978
9 TW	9.084856211928034
10 EA	9.171986970684038

MIA

uniquecarrier	AverageDepDelayMin
1 9E	0.5
2 PA (1)	4.84346374454242
3 EV	5.669603524229075
4 XE	6.1033769813921435
5 TZ	6.823035392921415
6 NW	6.9902354593253
7 US	7.427278231684071
8 ML (1)	7.6319514661274015
9 UA	8.273468482892824
10 PI	9.063902838987394

LAX

uniquecarrier	AverageDepDelayMin
1 PS	4.973895803502589
2 MQ	5.069745783395635
3 OO	6.09525787073169
4 ML (1)	7.101275318829708
5 NW	7.252479152149109
6 TZ	7.456864216054013
7 US	7.803737590192616
8 FL	8.082327701796729
9 F9	8.362138132928548
10 AA	8.41992740869826

IAH

uniquecarrier	AverageDepDelayMin
1 PI	4.643304503429764
2 PA (1)	5.73430303030303
3 NW	6.1597593951768665
4 WN	6.232248922121386
5 US	7.055723274437524
6 AA	7.269662304240027
7 TW	7.453365263423242
8 OO	7.943149703051403
9 HP	8.040625479074047
10 DL	8.277057959223324

SFO

uniquecarrier	AverageDepDelayMin
1 PA (1)	6.139633286318759
2 PS	6.438607283464567
3 TZ	7.678805535324107
4 DL	7.753307004627034
5 NW	7.945787665630624
6 MQ	8.114333300676007
7 US	8.576949681445846
8 TW	8.67769505033557
9 CO	8.875960421795885
10 F9	8.911214953271028

Question 2.3 Results: For each source-destination pair X-Y, rank the top-10 carriers in decreasing order of on-time arrival performance at Y from X.

CMI to ORD

uniquecarrier	AverageDelayMin
MQ	15.739150630391507

DFW to IAH

uniquecarrier	AverageDelayMin
UA	8.899408284023668
PA (1)	9.333333333333334
OO	9.736549165120593
CO	10.0064736160672
DL	10.204433400386542
EV	10.691978609625668
AA	12.147884747647687
XE	12.8929173693086
MQ	12.975917431192661

JFK to LAX

uniquecarrier	AverageDelayMin
UA	11.469386288506684
HP	14.865141955835963
AA	15.044821251483475
DL	16.631231597116457
PA (1)	17.09370780448285
TW	18.287762061126546

IND to CMH		ATL to PHX		LAX to SFO	
uniquecarrier	AverageDelayMin	uniquecarrier	AverageDelayMin	uniquecarrier	AverageDelayMin
CO	4.394163964798518	FL	12.61	PS	5.830402722631877
NW	7.601538461538461	US	12.687394957983193	TZ	6.238095238095238
US	7.838587981676098	HP	13.367140921409215	F9	6.965310206804537
HP	7.990588235294117	DL	13.867261117830722	US	10.821992785172284
AA	8.25	EA	14.008673469387755	MQ	10.933456561922366
DL	12.629807692307692			AA	12.465499230261711
EA	13.065420560747663			NW	12.79028697571744
				EV	13.39871382636656
				DL	13.483850453526124
				CO	14.001739130434782

Question 2.4: For each source-dest. pair X-Y, determine the mean arr. delay (in min.) for a flight from X to Y.

CMI to ORD

	AverageDelayMin
1	15.739150630391507

IND to CMH

	AverageDelayMin
1	6.613487475915222

DFW to IAH

	AverageDelayMin
1	11.060462025053555

LAX to SFO

	AverageDelayMin
1	13.695194295129438

JFK to LAX

	AverageDelayMin
1	14.632357646666017

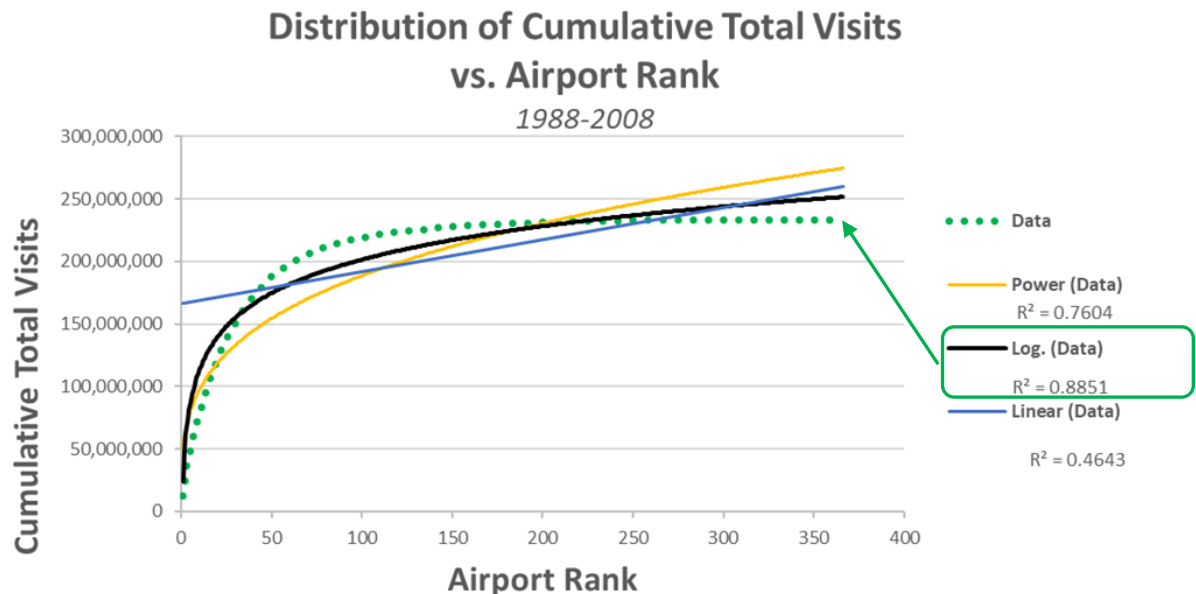
ATL to PHX

	AverageDelayMin
1	13.620285929786068

Task 1: Group 3 Questions

Question 3.1: Does the popularity distribution of airports follow a Zipf distribution? If not, what distribution does it follow?

No, it does not follow a Zipf 'power' distribution. As shown below, airport popularity most closely follows a log distribution.



Q3.1 Do My Results Make Sense & Are They Useful? Yes, the results make sense based on my knowledge of airline business practices. In short, a number of airports make up the vast majority of trips. This is due to US city population distributions as well as the way airlines set up 'hub' airports. These results are useful to help identify how critical the top ~25 airports (top 10%) are to the system overall: they make up 60%+ of visits. This can be used to potentially improve route congestion for future planning.

Q3.1 Approach & Algorithms: I used an Athena query to find total number of visits at each airport (350 airports, 230 million visits). I sent the results to a csv, then chose Excel to visualize/plot the data and find the best fitting distribution.

Q3.1 Application-Level Design Optimization: Excel's built-in distribution curve & plot capabilities- Selected Excel for its capability to quickly & easily add in distribution functions to compare to my data set.

Question 3.2: Find for each X-Y-Z and day/month (dd/mm) combination in the year 2008, the two flights (X-Y and Y-Z) that satisfy constraints (a) and (b) and have the best individual performance with respect to constraint (c), if such flights exist. Constraints:

- The second leg of the journey (flight Y-Z) must depart two days after the first leg (flight X-Y). For example, if X-Y departs on January 5, 2008, Y-Z must depart on January 7, 2008.
- Tom wants his flights scheduled to depart airport X before 12:00 PM local time and to depart airport Y after 12:00 PM local time.
- Tom wants to arrive at each destination with as little delay as possible. You can assume you know the actual delay of each flight.

Q3.2 Do My Results Make Sense & Are They Useful? Yes- the results make sense and matched exactly with the answers provided by the instructors. However, given the specificity of the flight constraints the results are not broadly useful.

Q3.2 Approach & Algorithms: Due to the size of the data set and CPU-intensive nature of the queries, this question required a different approach from the others in Groups 1 & 2. To make these queries more efficient from a run-time and resource perspective, I used the following approach:

- Use Athena to Create a Table In Glue with a Subset of the Total Data Set
 - This drastically reduced the number of records in the data set, making it much easier to work with. I filtered by the origins and travel dates specified in the question.
- Based on the Table Created in Step 1, Create a Joined Table of the Potential Flight Combinations
 - Using the smaller subset of data, I then performed a self-join function on the table. This built a table with the full combination of trips with 3 total locations.
- Perform Queries on the Joined Table to Find the Best Flight Combinations for Each of the 6 Trips
 - Finally, I performed queries on the joined table with the parameters outlined in the question. In order to match the instructor's exactly, I used 'arrdelay' to measure arrival performance for each leg of the trip. I then created another field called 'TotalArrivalDelay', which measured the sum of the two arrival delays in minutes.

Q3.2 Results

CMI → ORD → LAX, 04/03/2008

	FirstAirline ▼	FirstFlightNum ▼	SecondAirline ▼	SecondFlightNum ▼	TotalArrivalDelay ▼
1	MQ	4278	AA	607	-38.0

JAX → DFW → CRP, 09/09/2008

	FirstAirline ▼	FirstFlightNum ▼	SecondAirline ▼	SecondFlightNum ▼	TotalArrivalDelay ▼
1	AA	845	MQ	3627	-6.0

SLC → BFL → LAX, 01/04/2008

	FirstAirline ▼	FirstFlightNum ▼	SecondAirline ▼	SecondFlightNum ▼	TotalArrivalDelay ▼
1	OO	3755	OO	5429	18.0

LAX → SFO → PHX, 12/07/2008

	FirstAirline ▼	FirstFlightNum ▼	SecondAirline ▼	SecondFlightNum ▼	TotalArrivalDelay ▼
1	WN	3534	US	412	-32.0

DFW → ORD → DFW, 10/06/2008

	FirstAirline ▼	FirstFlightNum ▼	SecondAirline ▼	SecondFlightNum ▼	TotalArrivalDelay ▼
1	UA	1104	AA	2341	-31.0

LAX → ORD → JFK, 01/01/2008

	FirstAirline ▼	FirstFlightNum ▼	SecondAirline ▼	SecondFlightNum ▼	TotalArrivalDelay ▼
1	UA	944	B6	918	-6.0

Appendix

Appendix 1.1: Architecture

- **AWS:** <https://d1.awsstatic.com/whitepapers/lambda-architecture-on-for-batch-aws.pdf>

Overview

This section provides an overview of the various AWS services that form the building blocks for the batch, serving, and speed layers of lambda architecture.

Each of the layers in the Lambda architecture can be built using various analytics, streaming, and storage services available on the AWS platform.

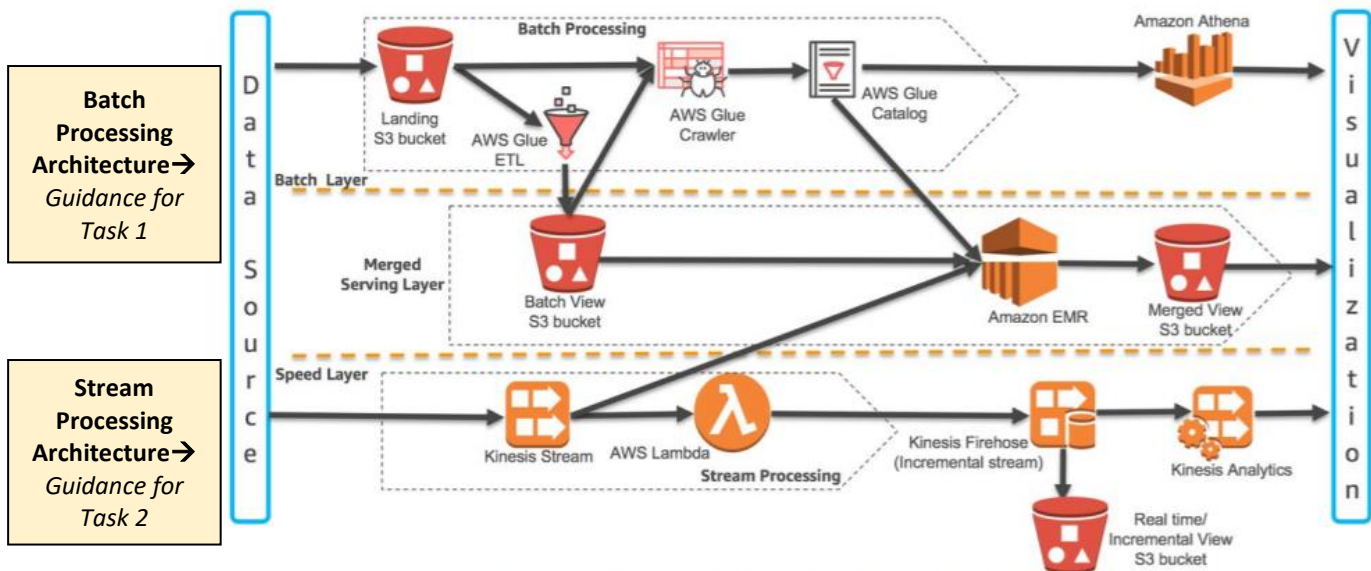


Figure 2: Lambda Architecture Building Blocks on AWS

Data Analysis

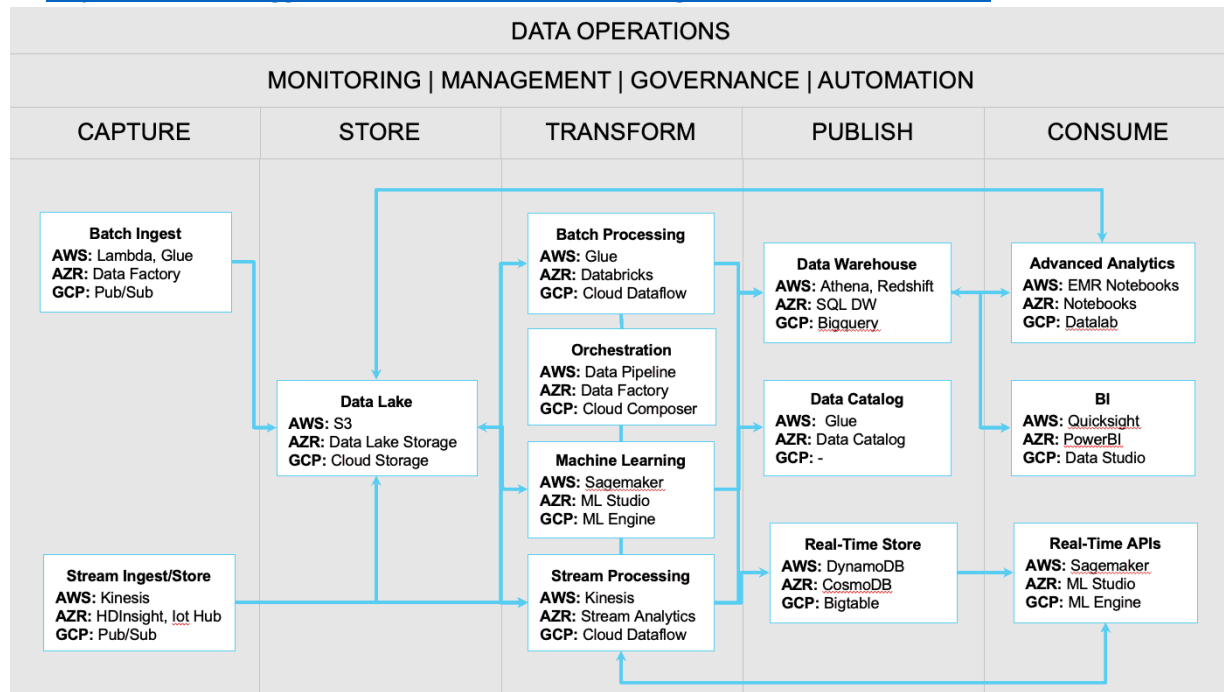
In this phase, you apply your query to analyze data in the three layers:

- **Batch Layer** – The data source for batch analytics could be the raw master data set directly or the aggregated batch view from the serving layer. The focus of this layer is to increase the accuracy of analysis by querying a comprehensive dataset across multiple or all dimensions and all available data sources.
- **Speed Layer** – The focus of the analysis in this layer is to analyze the incoming streaming data in near real-time and to react immediately based on the analyzed result within accepted levels of accuracy.
- **Serving Layer** – In this layer, the merged query is aimed at joining and analyzing the data from both the batch view from the batch layer and the incremental stream view from the speed layer.

This suggested architecture on the AWS platform includes Amazon Athena for the batch layer and Amazon Kinesis Data Analytics for the speed layer. For the serving layer, we recommend using Spark Streaming on an Amazon EMR cluster to consume the data from Amazon Kinesis Data Streams from the speed layer, and using Spark SQL on an Amazon EMR cluster to consume data from Amazon S3 in the batch layer. Both of these components are part of the same code base, which can be invoked as required, thus reducing the overhead of maintaining multiple code bases. The sample code that follows highlights using Spark SQL and Spark streaming to join data from both batch and speed layers.

- **Cloud Service Providers: Batch & Stream Processing**

- <https://www.kdnuggets.com/2019/06/understanding-cloud-data-services.html>



- **Other Architecture Reference Links**

- **AWS Glue**

- Overview: <https://aws.amazon.com/glue/?whats-new-cards.sort-by=item.additionalFields.postDateTime&whats-new-cards.sort-order=desc>
- Key Concepts: <https://docs.aws.amazon.com/glue/latest/dg/components-key-concepts.html>
- Parquet: <https://parquet.apache.org/documentation/latest/>
- Glue Apache Spark: <https://aws.amazon.com/blogs/big-data/best-practices-to-scale-apache-spark-jobs-and-partition-data-with-aws-glue/>
- ETL- Glue Apache Spark: <https://medium.com/tysonworks/managed-etl-using-aws-glue-and-spark-62a92292f19d>

- **AWS Athena**

- Overview: <https://aws.amazon.com/athena/?whats-new-cards.sort-by=item.additionalFields.postDateTime&whats-new-cards.sort-order=desc>
- Presto: <https://aws.amazon.com/big-data/what-is-presto/>

- **Other Big Data Ingestion / Architecture**

- <https://www.xenonstack.com/blog/big-data-ingestion/>

Appendix 1.2: Group 1 Queries

Q1.1

```
WITH t1 AS
  (SELECT DISTINCT origin
   FROM "flighttiming-db"."new_table")
SELECT t1.origin ,
       count(1) AS counter
FROM t1
JOIN "flighttiming-db"."new_table" AS t2
  ON t2.origin = t1.origin
  OR t2.dest = t1.origin
GROUP BY t1.origin
ORDER BY counter desc
LIMIT 10;
```

Q1.2

```
SELECT uniquecarrier,avg(try_cast(arrdelayminutes AS DOUBLE)) AS AverageDelayMin
FROM "flighttiming-db"."new_table"
GROUP BY uniquecarrier
ORDER BY avg(try_cast(arrdelayminutes AS DOUBLE)) asc
LIMIT 10;
```

Appendix 1.3: Group 2 Queries

Q2.1

```
SELECT uniquecarrier,avg(try_cast(depdelayminutes AS DOUBLE)) AS AverageDepDelayMin
FROM "flighttiming-db"."new_table"
WHERE origin='MIA'
GROUP BY uniquecarrier
ORDER BY avg(try_cast(depdelayminutes AS DOUBLE)) asc
LIMIT 10;
```

Q2.3

```
SELECT uniquecarrier,avg(try_cast(arrdelayminutes AS DOUBLE)) AS AverageDelayMin
FROM "flighttiming-db"."new_table"
WHERE origin='CMI' AND dest='ORD'
GROUP BY uniquecarrier
ORDER BY avg(try_cast(arrdelayminutes AS DOUBLE)) asc
LIMIT 10;
```

Q2.4

```
SELECT avg(try_cast(arrdelayminutes AS DOUBLE)) AS AverageDelayMin
FROM "flighttiming-db"."new_table"
WHERE origin='ATL' AND dest='PHX';
```

Appendix 1.4: Group 3 Queries

Q3.1

```
WITH t1 AS
  (SELECT DISTINCT origin
   FROM "flighttiming-db"."new_table")
SELECT t1.origin ,
       count(1) AS counter
FROM t1
JOIN "flighttiming-db"."new_table" AS t2
  ON t2.origin = t1.origin
  OR t2.dest = t1.origin
GROUP BY t1.origin
ORDER BY counter desc;
```

Results

	origin ▼	counter ▼
1	ORD	12449353
2	ATL	11540422
3	DFW	10799303
4	LAX	7723595
5	PHX	6585534
6	DEN	6273787
7	DTW	5636622
8	IAH	5480734
9	MSP	5199213
10	SFO	5171033
11	EWR	5136971
12	STL	5125926

Q3.2

Step 1: Create Table with Filtered Data Set

```
CREATE TABLE IF NOT EXISTS "key2008placesdates" AS
SELECT flightdate,
       uniquecarrier,
       flightnum,
       origin,
       dest,
       crsdeptime,
       deptime,
       depdelayminutes,
       arrtime,
       arrdelay,
       arrdelayminutes,
       cancelled
FROM "flighttiming-db"."allflightscs598aviationbucket"
WHERE flightdate in('2008-03-04','2008-03-06','2008-09-09','2008-09-11','2008-04-01','2008-04-03','2008-07-12','2008-07-14','2008-06-10','2008-06-12','2008-01-01','2008-01-03')
       AND origin in('CMI','ORD','JAX','DFW','SLC','BFL','LAX','SFO');
```

Step 2: Create Joined Table

```
CREATE TABLE IF NOT EXISTS "joinedflights2008" AS
WITH t1 AS
  (SELECT *
   FROM "flighttiming-db"."key2008placesdates")
SELECT t1.uniquecarrier AS xyuniquecarrier,
       t1.flightnum AS xyflightnum,
       t1.flightdate AS xyflightdate,
       t1.crsdeptime AS xycrsdeptime,
       t1.arrdelay AS xyarrdelay,
       t1.origin AS xflight,
       t1.dest AS yflight,
       t2.dest AS zflight,
       t2.uniquecarrier AS yzuniquecarrier,
       t2.flightnum AS yzflightnum,
       t2.flightdate AS yzflightdate,
       t2.crsdeptime AS yzcrsdeptime,
```

```

        t2.arrdelay AS yzarrdelay
FROM t1
JOIN "flighttiming-db"."key2008placesdates" AS t2
    ON t2.origin = t1.dest
    AND t1.flightdate !=t2.flightdate
GROUP BY t1.uniquecarrier, t1.flightnum,t1.flightdate, t1.crsdeptime, t1.arrdelay, t1.origin, t1.dest, t2.uniquecarrier,
t2.flightnum,t2.dest, t2.flightdate, t2.crsdeptime, t2.arrdelay;

```

Step 3: Perform Queries

```

SELECT xyuniquecarrier AS FirstAirline,
       xyflightnum AS FirstFlightNum,
       yzuniquecarrier AS SecondAirline,
       yzflightnum AS SecondFlightNum,
       try_cast(xyarrdelay AS DOUBLE) + try_cast(yzarrdelay AS DOUBLE) AS TotalArrivalDelay,
       *
FROM "flighttiming-db"."joinedflights2008"
WHERE xflight='JAX'
      AND yflight='DFW'
      AND zflight='CRP'
      AND xyflightdate='2008-09-09'
      AND yzflightdate='2008-09-11'
      AND try_cast(xycrsdeptime AS INTEGER)<1200
      AND try_cast(yzcrsdeptime AS INTEGER)>1200
ORDER BY TotalArrivalDelay ASC LIMIT 1;

```